

CONTENTS

- Overview
- 1 The questions evaluations exist to answer
- 2 The three core challenges of LLM development
- 3 What an eval is, and the analyze-measure-improve cycle
- 4 Level 1: unit tests as evals
- 5 Level 2: humans first, then an aligned model judge
- 6 The alignment process and the agreement score
- 7 Level 3: A/B testing with real users
- 8 Reference-based versus reference-free metrics
- 9 Four mistakes, and the principles that avoid them
- Glossary
- Check yourself
- Where to go next

The Three Levels of LLM Evals: Unit Tests, Aligned Judges & A/B Testing

Build an evaluation-driven workflow — assertions on every change, human-aligned judges every week, and A/B tests on major releases.

For engineers building production LLM applications who want a systematic way to improve them instead of guessing · 27 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)[Read the full lesson](#)[Read the highlights](#)

BEFORE YOU START

- Comfortable writing Python, including functions and exceptions
- You have built at least one LLM workflow with a prompt and structured output
- Familiarity with the idea of tracing or logging requests in production

Overview

Reports suggest a large share of AI projects fail to reach production — Gartner has projected that 40% of agentic AI projects will be cancelled by 2027, and an MIT figure of 95% of generative AI pilots failing has circulated widely. Whether or not those exact numbers hold, the pattern behind them is real: teams build something that works on the happy path, ship it, and then have no systematic way to find out what is going wrong or to fix it without breaking something else.

This lesson lays out the engineering answer. It starts with the three core challenges that make LLM development different from ordinary software — you do not understand the data flowing through your system, you cannot fully specify what you want in a prompt, and behaviour is inconsistent across near-identical inputs. It then presents evaluations organised into three levels of increasing cost: unit tests you run on every change, human and model evaluation you run weekly, and A/B tests you run on major releases.

The centrepiece is the alignment process: how to build an LLM judge that actually agrees with a domain expert, measured by an agreement score you drive upward through iteration. This is the step nearly everyone skips in favour of an off-the-shelf tool with a pretty dashboard, and skipping it is what produces confident metrics that mean nothing.

KEY TAKEAWAYS

- ✓ Three core challenges define LLM development: not understanding your data at scale, the gap between what you want and what you can specify, and inconsistent behaviour on similar inputs.
- ✓ An eval is a systematic measurement of quality — a single metric for a specific aspect. One system has many evals.
- ✓ Evals are operationalised three ways: background monitoring, real-time guardrails, and improvement tooling for labelling and fine-tuning data.
- ✓ The analyze → measure → improve cycle is the working loop: collect and categorise failures, turn them into metrics, then change prompts, models, or architecture.
- ✓ Level 1 is unit tests — fast, cheap assertions run on every code and prompt change.
- ✓ Level 2 is human and model evaluation — systematic review run weekly or bi-weekly. Humans must come first; the LLM judge is aligned to them, never the reverse.
- ✓ Level 3 is A/B testing with real users, reserved for major releases because of its cost.
- ✓ The agreement score between human and model critique is the metric you optimise; meta-prompting can rewrite the judge prompt to raise it.
- ✓ Reference-based metrics compare to a known correct answer; reference-free metrics score qualities like tone or safety where many answers are valid.
- ✓ The four common mistakes are tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges.

01 The questions evaluations exist to answer

2:04

Every engineer building on LLMs eventually hits a wall of questions that logs alone cannot answer.

KEY POINTS

- Tracing platforms let you diagnose issues reactively, after they surface — necessary but not sufficient.
- The hard question is regression: does fixing one case break others you are not watching?
- LLMs are non-deterministic, context-sensitive, and a response can be factually correct while the tone is wrong.
- A question can have many valid answers, and failure modes are subtle and hard to predict.
- An LLM application is never truly finished — there is always a further prompt optimisation available.

The regression you cannot see

A customer reports that billing tickets are misrouted. You add a line to the system prompt emphasising billing keywords, confirm the reported ticket now routes correctly, and ship. Two weeks later, technical tickets containing the word 'charge' — as in 'my device will not charge' — are being sent to billing. Nothing alerted you, because you only ever re-checked the case you were fixing.

02 The three core challenges of LLM development

4:52

Understanding your data, closing the specification gap, and coping with inconsistent behaviour — these are what evaluation exists to address.

KEY POINTS

- Understanding the data: you test the happy path; real usage at scale is drastically different and users are unpredictable.
- The specification gap: you recognise a good output but cannot fully express what makes it good in a prompt.
- Attribution is part of the gap — was the failure prompting, retrieval, or pipeline logic?
- Inconsistent behaviour: small wording changes produce completely different outputs.
- Success rests on three capabilities — evaluation quality, debugging, and the ability to change behaviour. Most teams build only the last.

Attribution in a RAG pipeline

A customer gets a wrong answer about a refund window. There are at least three distinct causes with three distinct fixes, and you cannot tell them apart without instrumenting each stage.

Bad answer observed

```
|  
+-- Retrieval returned the wrong policy document?  
|    -> fix chunking / embeddings / reranking  
|  
+-- Right document retrieved, model ignored it?  
|    -> fix the prompt's grounding instructions  
|  
+-- Right document, right reading, wrong arithmetic?  
    -> move the computation out of the model
```

03 What an eval is, and the analyze-measure-improve cycle

9:01

An eval is a systematic measurement of one specific quality aspect; the working loop is collect failures, turn them into metrics, then improve.

KEY POINTS

- An eval is a systematic measurement of quality; a single eval measures one specific aspect, and a system has many.
- Three operationalisations: background monitoring, real-time guardrails, and improvement tooling for labelling and datasets.
- Analyze: collect real examples and categorise failure modes from complaints, data inspection, and application errors.
- Measure: turn insights into quantitative metrics — booleans first, scores and rankings later.
- Improve: refine prompts, swap models, change architecture — then repeat the cycle.

One system, many evals

A single support workflow carries several independent measurements, each answering a different question about the same response.

```
evals = {  
    "format_valid":      lambda r: r.category in CATEGORIES,  
    "has_response":      lambda r: len(r.response) > 10,  
    "factually_correct": judge_factual,      # LLM judge  
    "tone_appropriate":  judge_tone,         # LLM judge  
    "no_pii_leaked":     regex_pii_check,    # guardrail  
}
```

04 Level 1: unit tests as evals

12:26

Fast, cheap Python assertions over structured output, run on every code and prompt

change.

KEY POINTS

- Fast, cheap assertions run on every code and prompt change — the tier you run most often.
- Python's `assert` passes silently on true and raises `AssertionError` on false.
- Structured output is the natural target: assert on allowed values, types, and ranges.
- Keep an `evals` folder of raw event JSON captured from your real system — this is why capturing raw events matters.
- Assert at every step of a multi-step workflow, not just the final output.
- What to test comes from your failure modes: when something breaks, write the test that catches it next time.

Assertions over a classification step

Load a real captured event, run the workflow, and assert the properties that must hold. The generic checks catch schema drift; the specific check pins this ticket's expected routing.

```
import json
from pathlib import Path

CATEGORIES = {"billing", "technical", "general"}

def load_event(name):
    return json.loads(Path(f"evals/events/{name}.json").read_text())

def test_billing_categorization():
    event = load_event("billing_double_charge")
    result = process_customer_message(event["message"])

    # generic invariants – catch schema drift
    assert result.category in CATEGORIES
    assert isinstance(result.confidence, float)
    assert 0.0 <= result.confidence <= 1.0
    assert len(result.response) > 10

    # input-specific expectation
    assert result.category == "billing"
```

The common misconception is that you can hand evaluation straight to an LLM. You cannot instruct a model to review what you have not learned to review yourself.

KEY POINTS

- Run systematic review weekly or bi-weekly; it costs more than level 1 because humans spend time on it.
- You cannot skip the human step — you cannot write instructions for a review you have not learned to perform.
- Involve both engineers (to understand the data flow) and domain experts (to know what good means).
- Remove all friction from viewing data; a spreadsheet is a legitimate and effective tool.
- Use the most capable model you can afford as the judge, focus on specific dimensions, and start binary.
- Always generate detailed critiques, not just scores.
- Drift is permanent — what good looks like changes, so track human-model agreement continuously.

A judge prompt with a required critique

Binary outcome, explicit criteria, and an explanation the human can audit. Start here rather than with a five-metric scoring rubric.

```
JUDGE = """Evaluate this customer service response on these criteria:
```

- accuracy: are the facts correct?
- helpfulness: does it solve the customer's problem?
- tone: is it professional and empathetic?

```
<question>{question}</question>
```

```
<response>{response}</response>
```

```
Provide a detailed critique explaining your reasoning,  
then rate 1 (good) or 0 (bad).
```

```
"""
```

```
class Critique(BaseModel):  
    explanation: str  
    outcome: bool
```

06 The alignment process and the agreement score

22:47

Run model critique and human critique in parallel over the same rows, score the match, and iterate the judge prompt until agreement climbs.

KEY POINTS

- The loop: model critiques and human critiques on the same rows, compared, with the judge prompt iterated until they agree.
- The human critique is the fixed golden standard; the judge prompt is what you change.
- Agreement score = the fraction of rows where the human and model outcomes match.
- Meta-prompting — showing a strong model the mismatches and the current judge prompt, and asking it to rewrite for higher agreement — is the powerful way to iterate.
- Real captured inputs are best, but LLM-generated synthetic examples are a legitimate starting point.
- This resembles ML experimentation more than software engineering: move a metric through repeated experiments.

The alignment spreadsheet

Ten rows, two independent critiques, and one derived column. The 70% at the bottom is the number every iteration tries to raise.

#	input	output	model	human	match
1	order 3 days late	I understand...	1	1	1
2	cancel subscription	Sure, here...	1	1	1
3	charged twice	Let me check...	1	0	0
4	where is my refund	Refunds take...	0	0	1
5	wrong size sent	Apologies...	1	1	1
...					

agreement score = 7/10 = 70%

07 Level 3: A/B testing with real users

31:46

The trickiest and most expensive level — compare prompts, models, or whole workflows by their effect on real business outcomes.

KEY POINTS

- Level 3 measures business impact with real users, reserved for major releases because it is costly.
- It requires enough data to be meaningful — a handful of examples is not an experiment.
- A cheaper middle ground: compare prompt variants against your own labelled set and its good/bad rate.
- Full in-app A/B testing with satisfaction ratings is complex and suits mature, high-traffic applications.
- Metrics here are business metrics: satisfaction, task completion, time to resolution, engagement, retention.

Offline prompt comparison before any user sees it

Run two prompt variants over the same fixed dataset and compare their good rates using the judge you already aligned. Most decisions can be made here.

```
def good_rate(prompt, dataset, judge):
    outputs = [run_workflow(row["input"], prompt) for row in dataset]
    verdicts = [judge(o) for o in outputs]
    return sum(v.outcome for v in verdicts) / len(verdicts)

baseline = good_rate(PROMPT_V1, dataset, aligned_judge) # 0.60
variant = good_rate(PROMPT_V2, dataset, aligned_judge) # 0.75

print(f"baseline {baseline:.0%} -> variant {variant:.0%}")
```

08 Reference-based versus reference-free metrics

34:29

When you know the correct answer, testing is easy; when many answers are valid, you score qualities instead.

KEY POINTS

- Reference-based: you know the correct answer — exact match, semantic similarity, code execution, SQL correctness, schema validation.
- Much reference-based checking can live at level 1 as unit tests.
- Reference-free: many outputs are valid — tone, length, hallucination, format compliance, safety and toxicity.
- Reference-free metrics are usually scored on a 0–1 or 1–5 scale rather than pass/fail.
- Real systems lean heavily on reference-free metrics, which is why judge alignment matters.
- Whichever you use, start binary — good and bad is hard enough to get right.

Both kinds on one response

The same support reply carries reference-based checks that are trivially automatable and reference-free ones that need a judge.

```
# reference-based – a known correct value exists
assert result.category == "billing"
assert result.order_id == "A-10294"
assert json.loads(result.payload)          # schema valid

# reference-free – many valid answers, score the quality
tone      = judge_tone(result.response)    # 0.0 – 1.0
grounded  = judge_no_hallucination(result)  # bool
safe      = judge_safety(result.response)   # bool
```

09 Four mistakes, and the principles that avoid them

37:12

Tool-first thinking, generic-metrics obsession, avoiding your data, and unaligned judges — with looking at data as the highest-value habit.

KEY POINTS

- Tool-first thinking: buying a database, a model, or a platform instead of diagnosing. Fix: start simple, build what you need.
- Generic-metrics obsession: 'helpfulness 4.2' is unactionable. Fix: specific metrics, starting binary.
- Avoiding your data: the most consequential mistake. Fix: look at lots of real data, constantly and forever.
- Unaligned judges: assuming the judge works. Fix: validate alignment with a human first.
- Manual inspection has the highest value-to-prestige ratio — unglamorous and the most useful thing you can do.
- Only use LLMs to scale: generate test cases, create synthetic data, automate critique — but always validate with humans.

You are succeeding versus you are struggling

The concrete symptoms of an evaluation-driven workflow, contrasted with its absence. This is the diagnostic to run on your own team.

Succeeding

Deploy changes confidently
Failures caught before users
You understand system behaviour
Improvements compound over time

Struggling

Everything breaks something else
Surprised by user complaints
Progress feels like trial & error
You cannot measure if changes help

Glossary

Eval

A systematic measurement of quality in an AI system; a single eval measures one specific aspect of performance.

Evaluation-driven workflow

A development process where improvement is systematic rather than accidental, because every change is measured against a fixed set of evals.

Analyze–measure–improve

The evaluation lifecycle: collect and categorise failures, turn insights into quantitative metrics, then refine prompts, models, or architecture.

Specification gap

The distance between what you want a system to do and what you can actually express in prompts and code.

Assertion

A Python statement that passes silently when its condition is true and raises an `AssertionError` when false, used as the basis of level 1 evals.

Agreement score

The fraction of examples where a human's critique outcome matches the LLM judge's outcome; the metric you iterate the judge prompt to improve.

Meta-prompting

Asking a strong model to rewrite your judge prompt by showing it the human-model disagreements and the current prompt, so it encodes what the human cares about.

Reference-based metric

A measurement made against a known correct answer — exact match, semantic similarity, code execution, or schema validation.

Reference-free metric

A measurement of a quality where many outputs are valid — tone, length, hallucination, format compliance, safety.

Guardrail

An eval operationalised in real time to block a bad output before it reaches the user.

Raw event

The exact input payload your workflow received, captured and stored so it can later be replayed as a test fixture.

Drift

The gradual change over time in what 'good' means for your system, caused by prompt changes, data changes, new features, or different users.

Check yourself

Answer first, then open each one.

Why can you not build an LLM judge before doing human evaluation, even though the judge would save time?

Because the judge prompt is a written specification of what to look for, and you cannot write that specification for a review you have not learned to perform. Until a human — ideally a domain expert — has read real data and formed a view of what good means, you have nothing to instruct the model with, and any metrics it produces will look rigorous while measuring the wrong thing.

During the alignment loop, why does the human critique stay fixed while the judge prompt changes?

Because the human critique is the golden standard you are optimising toward. If you adjusted the human labels to match the model, you would be moving the target to meet the measurement, which guarantees agreement while destroying its meaning. Only the judge prompt is a free variable.

Why is capturing raw events from production described as crucial for level 1 evals?

Because raw events are your test fixtures. Assertions need realistic inputs, and inputs you invent reflect the happy path you already imagined rather than the messy reality users produce. Stored raw events let you replay exactly what your system received when it failed, turning each incident into a permanent regression test.

Your judge reports 'helpfulness 4.2' across all outputs. Why is this an example of generic-metrics obsession, and what would be better?

Because the number is not actionable — it does not tell you what is wrong or what to change, and small movements in it are indistinguishable from noise. A better approach is specific binary metrics tied to failure modes you have actually observed, such as 'did it give the customer a route to resolution?', which points directly at a fix when it fails.

Why does the lesson say human and model evaluation must continue permanently rather than being finished after a few weeks of alignment?

Because of drift. Prompt modifications, data changes, new features, and different users all change what good looks like over time, so a judge aligned to last quarter's standard slowly stops measuring the current one. Tracking human-model agreement continuously is what detects that divergence before it corrupts your metrics.

You fix a misrouted billing ticket by editing the system prompt and confirm the fixed case now works. Why is this insufficient, and what does the three-level structure add?

Because confirming the case you fixed says nothing about regressions elsewhere — the prompt change may have broken cases you are not looking at. Level 1 gives you a suite of assertions over captured events that you re-run after every change, so 'I fixed it' becomes 'I fixed it and the other cases still pass'.

Where to go next

- Create an evals/events/ folder, drop in ten real captured payloads from your system, and write assertions for the structured fields each one should produce.
- Run the alignment exercise on ten rows: judge critique in one column, your own critique in another, and compute the agreement score.
- Take your worst-disagreeing rows and write the meta-prompt that rewrites your judge prompt, then measure whether agreement actually improved.
- Convert your ad-hoc test loop to pytest so the whole eval folder runs from a single terminal command.
- Audit your current metrics: for each one, ask what you would change if it moved by 10%. Delete the ones with no answer.
- Block out time to read production traces with no goal other than understanding what users actually send you.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.